COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

23IT723 - DEVOPS LABORATORY

PROJECT

LASTZONE – ONLINE TOURNAMENT PLATFORM



OCTOBER 2026

Submitted by

230371762052

1014 DHARNIDHARAN V R

COIMBATOREINSTITUTEOFTECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled "**LASTZONE – ONLINE TOURNAMENT PLATFORM**" is the bonafide work of 230371762052101422022-DHARNIDHARAN V completed during the **academic year 2025-2026-Semester VII** for the project presentation of **23IT723 - DEVOPS LABORATORY** under our supervision.

Certified that the candidate was examined by using the project work examination.

Review Members:

1. **Dr.M.Sangeetha**
2. **Dr.E.Arul**

23IT723 – DEVOPS LABORATORY

FINAL MINI PROJECT – EVALUATION SCHEME

Final Mini Project Assessment

The final mini project shall require students to integrate the major DevOps practices covered in the laboratory. The project should demonstrate practical implementation of version control, collaborative development, Continuous Integration, containerization and/or automated configuration/provisioning, with proper execution evidence.

S.No.	Evaluation Component	Key Verification Points	Marks
1	Problem Definition & Project Planning	Problem statement, objectives, requirements, scope, module identification, technology selection and workflow planning.	10
2	Git Repository & Version Control	Repository creation, meaningful commits, README, .gitignore, remote repository, push/pull/fetch operations and commit history.	15
3	Branching, Merging & Collaboration	Feature branches, parallel development, merging, conflict handling, branch organization and collaborative workflow.	10
4	Application Development & Build	Working application/source code, project structure, dependency management, build configuration and successful execution.	15
5	Jenkins Continuous Integration	Jenkins configuration, source-code integration, automated checkout, build job/pipeline, build execution and console verification.	15
6	Docker / Containerization	Dockerfile, image creation, container execution, port/configuration handling, application deployment and container verification.	10
7	Automation / Configuration Management	Ansible/Chef/Puppet/SaltStack or equivalent automation where applicable; repeatable configuration/provisioning and successful execution.	5
8	Testing & Verification	Functional testing, build verification, error identification, test evidence and corrective action.	5
9	Documentation &	Architecture/workflow diagram,	5

	Technical Presentation	commands/configuration, screenshots, results, troubleshooting, conclusion and repository details.	
10	Final Demonstration & Viva-Voce	Live demonstration, explanation of implementation, troubleshooting responses and technical understanding.	10
		TOTAL	**100**

Recommended Mark Distribution within the Final Demonstration

For uniform evaluation, the examiner may use the following internal split for the implementation and demonstration components.

Assessment Area	Marks	Awarded Marks	Remarks
Working implementation and correctness	30		
DevOps tool integration and automation	20		
Version control and collaborative workflow	15		
Build / deployment / container verification	15		
Testing and troubleshooting	5		
Documentation and technical presentation	5		
Live demonstration and viva-voce	10		
TOTAL	**100**		

Minimum Evidence to be Submitted

- Project repository URL and project README.
- Git commit history and branch/merge evidence.
- Jenkins job/pipeline configuration and successful build evidence.
- Dockerfile, image and running container evidence where Docker is used.
- Automation/configuration scripts where applicable.
- Application execution and testing evidence.
- Architecture/workflow diagram.
- Short technical report containing implementation steps, results and troubleshooting.
- Live demonstration of the complete DevOps workflow.

Evaluation Guidelines

- Marks shall be awarded primarily for a working implementation and demonstrable output.
- Copied source code without functional understanding shall not receive full marks.
- Screenshots may support the evaluation, but critical components should be demonstrated live wherever possible.
- Students should use meaningful repository names, branch names and commit messages.
- If a component is not applicable to the selected project, the student shall obtain faculty approval and demonstrate an equivalent DevOps automation component.
- The final viva shall focus on the student's own implementation, commands used, tool configuration and troubleshooting.

Faculty Evaluation Record

Student Name	DHARNIDHARAN V	**Register No.**	2303717620521014
Project Title	LastZone – Online Tournament Platform	**Date**	
Final Mark			
Faculty Evaluator I		**Signature**	
Faculty Evaluator II	/ 100	**Signature**	

TABLE OF CONTENTS

S. NO.	CONTENT	PAGE NO.
1	INTRODUCTION	05
2	OBJECTIVES	05
3	ABSTRACT	06
4	TECHOLOGY STACK	06
5	SYSTEM ARCHITECTURE	07
6	MODULE 1 —USER INTERFACE & zombie survival	08

	tournament gameplay	
7	MODULE 2 — Spring Boot 3.3.5 BACKEND & score and run management	09
8	MODULE 3 — PostgreSQL 16 DATABASE MANAGEMENT	10
9	MODULE 4 — GIT & GITHUB SOURCE CONTROL	11
10	MODULE 5 — DOCKER CONTAINERIZATION	12
11	MODULE 6 — JENKINS CI/CD PIPELINE	14
12	MODULE 7 — AUTOMATED TESTING	15
13	MODULE 8 — ANSIBLE DEPLOYMENT	17w
14	MODULE 9 — PERSISTENT STORAGE	18
15	MODULE 10 — ADMIN & score and run management	19
16	MODULE 11 — CI/CD INTEGRATION	20
17	MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION	21
18		23

	RESULTS AND OUTPUT	
19	COMMANDS USED	23
20	ADVANTAGES	28
21	LIMITATIONS	28
22	FUTURE ENHANCEMENTS	29
23	CONCLUSION	29

1. INTRODUCTION

The LASTZONE – ONLINE TOURNAMENT PLATFORM is a web-based zombie survival tournament gameplay and management system developed to simplify the process of ordering food in a tournament environment. The system allows users to browse available leaderboard entrys, search for dishes, select leaderboard entrys, add them to a cart, and place orders through a digital interface. An leaderboard dashboard is also provided to view and manage placed orders.The application is developed using HTML, CSS, JavaScript, Java 21, Spring Boot 3.3.5, and PostgreSQL 16, with Spring Boot 3.3.5 handling backend operations and PostgreSQL 16 storing order information. The project also incorporates DevOps practices to automate the development, testing, and deployment process. Git and GitHub are used for source code management, while Jenkins automates the CI/CD pipeline. Docker provides

application containerization, Spring Boot Test + MockMvc (JUnit 5) performs automated testing, and Ansible automates deployment and container configuration. A Docker Volume is used to provide persistent storage for the PostgreSQL 16 database.The project combines a functional zombie survival tournament gameplay system with DevOps automation to provide a structured and reliable approach to application development and deployment.

2.OBJECTIVES

- To develop a digital zombie survival tournament gameplay system that enables users to browse, search, select leaderboard entrys, add them to a cart, and place orders conveniently.
- To implement an leaderboard dashboard for viewing placed orders and maintaining order information using PostgreSQL 16.
- To organize order data systematically and provide quick access to stored order details through the leaderboard dashboard.
- To implement DevOps practices throughout the application development and deployment process using Git, GitHub, and Jenkins.
- To automate the CI/CD workflow using Jenkins for building, testing, and deploying the application.
- To containerize the application using Docker to provide a consistent and portable execution environment.
- To integrate Spring Boot Test + MockMvc (JUnit 5) for automated testing to verify application functionality and reduce errors before deployment.
- To automate application deployment and container configuration using Ansible.
- To implement Docker Volume for persistent storage of the PostgreSQL 16 database and ensure that order information is retained even when the application container is recreated.

3. ABSTRACT

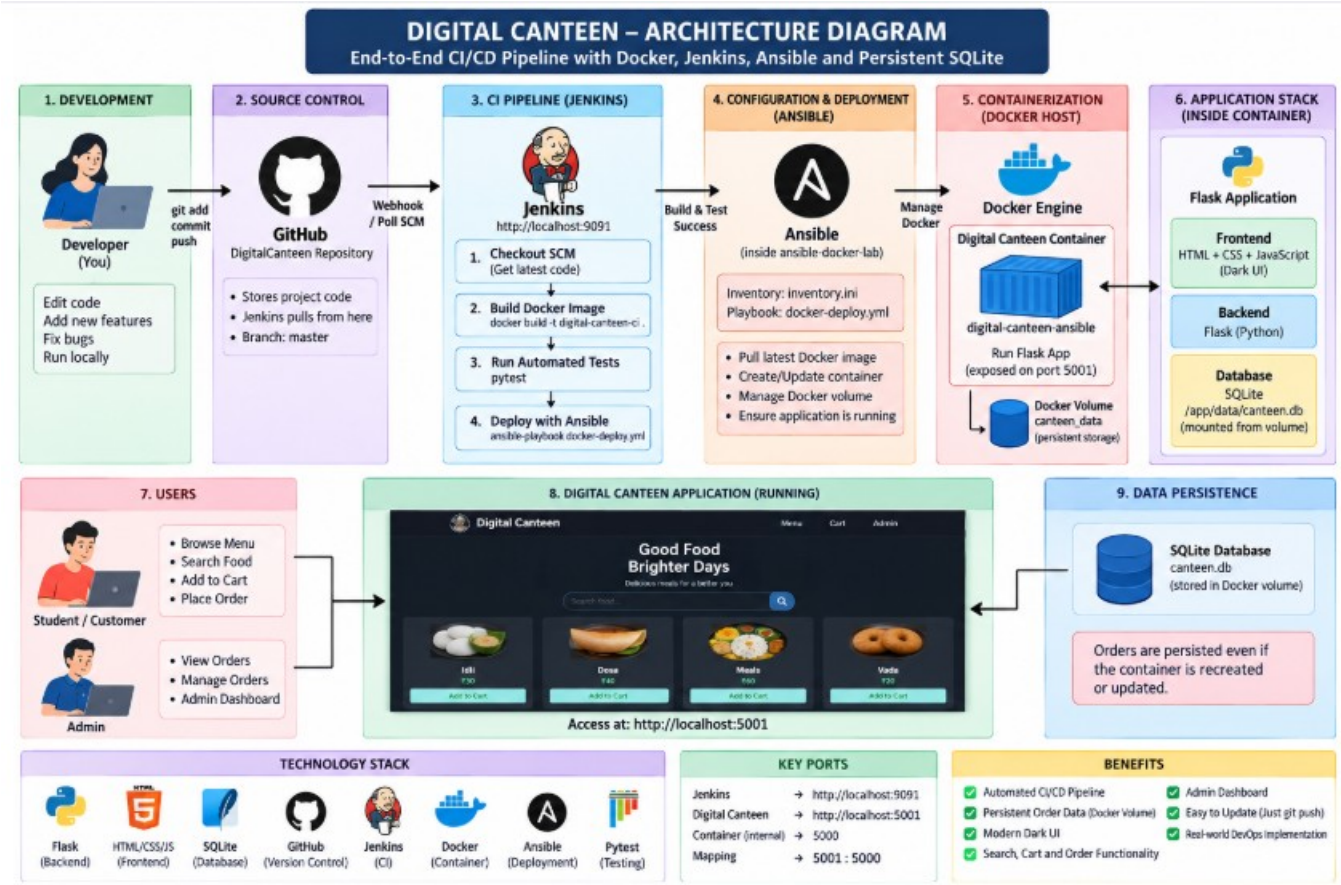
The LASTZONE – ONLINE TOURNAMENT PLATFORM is a web-based zombie survival tournament gameplay and management system designed to simplify the process of ordering food in a tournament environment. Traditional tournament operations may involve long queues, manual order processing, difficulty in managing multiple orders, and challenges in maintaining order records. The proposed system provides a digital platform where users can browse available leaderboard entrys, search for dishes, select leaderboard entrys, add them to a cart, and place orders conveniently. An administrative dashboard is also provided for viewing and managing placed orders. The application is developed using HTML, CSS, JavaScript, Java 21, Spring Boot 3.3.5, and PostgreSQL 16, with Spring Boot 3.3.5 handling backend operations and PostgreSQL 16 storing order information. The project also implements DevOps practices to automate the software development, testing, and deployment process. Git and GitHub are used for version control and source code management, while Jenkins automates the CI/CD pipeline. Docker is used to containerize the application and provide a consistent execution environment, Spring Boot Test + MockMvc (JUnit 5) performs automated testing, and Ansible automates application deployment and container configuration. A Docker Volume named **lastzone_postgres_data** is used to provide persistent storage for the PostgreSQL 16 database, ensuring that stored order information is retained even when the application container is recreated. The complete workflow follows **GitHub → Jenkins → Docker Build → Spring Boot Test + MockMvc (JUnit 5) → Ansible → Docker Container → Spring Boot 3.3.5 → PostgreSQL 16**, demonstrating the integration of a functional web application with modern DevOps practices and persistent data management.

4.TECHNOLOGY STACK

Technology	Version	Purpose
Java (OpenJDK)	21 LTS	Backend programming language
Spring Boot	3.3.5	REST API backend framework
React + TypeScript	18 / TS 5.x	Frontend web framework
Vite	5.x	Build tool & dev server
PostgreSQL	16-alpine	Production relational database
Unity WebGL	2022.3 LTS	3D game engine (396

Technology	Version	Purpose
		MB)
Nginx	1.25-alpine	Web server & reverse proxy
Docker	27.x	Container runtime
Docker Compose	v2.x	Service orchestration
Jenkins	2.462 LTS	CI/CD pipeline automation
Ansible	Core 2.17	Infrastructure-as-code
Caddy	2.x	HTTPS reverse proxy
JWT + BCrypt	jjwt 0.11	Authentication & security

5.SYSTEM ARCHITECTURE



1. Development: Developer creates and updates the LASTZONE – ONLINE TOURNAMENT PLATFORM application.

2. Source Code Management: The application source code and configuration files are maintained using Git.
3. GitHub Repository: The updated code is pushed to the GitHub repository for centralized source code management.
4. Jenkins Trigger: Jenkins retrieves the latest source code from the GitHub repository and starts the CI/CD pipeline.
5. Docker Build: Jenkins builds the application into a Docker image containing the Spring Boot 3.3.5 application and required dependencies.
6. Automated Testing: Jenkins runs Spring Boot Test + MockMvc (JUnit 5) to verify the application before deployment.
7. Deployment Automation: After successful testing, Jenkins invokes Ansible to automate the deployment process.
8. Container Configuration: Ansible creates and configures the Docker container with the required image, port mapping, restart policy, and volume configuration.
9. Application Execution: The Docker container runs the Spring Boot 3.3.5 backend, HTML/CSS/JavaScript frontend, and PostgreSQL 16 database.
10. User Interaction: Students can browse leaderboard entrys, search for dishes, add items to the cart, and place orders through the application.
11. Admin Management: The leaderboard dashboard allows administrators to view and monitor the placed order information stored in the database.

MODULE 1 — USER INTERFACE & zombie survival tournament gameplay

This module provides the user-facing interface of the LASTZONE – ONLINE TOURNAMENT PLATFORM application. It is designed to make food browsing and ordering simple and convenient for students.

Main Features

- Displays available leaderboard entrys with relevant details.
- Provides a search option to find leaderboard entrys quickly.
- Organizes leaderboard entrys into different categories.
- Allows users to select individual leaderboard entrys.
- Allows selected items to be added to the game run session.
- Provides cart management for selected leaderboard entrys.
- Displays the selected items and order details before confirmation.
- Allows users to place the final order.

- Provides order confirmation after successful submission.
- Provides a simple and user-friendly interface for accessing tournament services.
- Uses responsive interface elements for better usability.
- Connects the user interface with the Spring Boot 3.3.5 backend for processing requests.

Technologies: HTML, CSS, JavaScript.

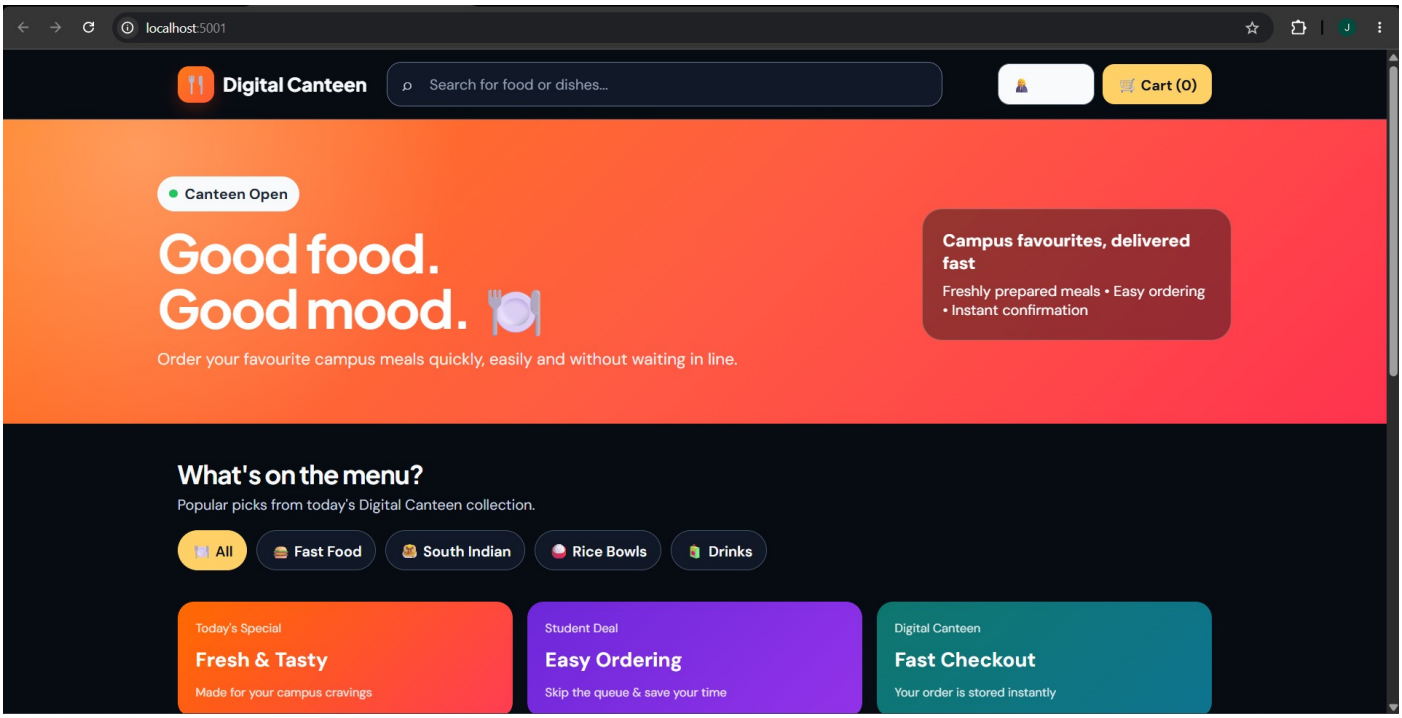


Figure 7.1: LASTZONE – ONLINE TOURNAMENT PLATFORM Home Page

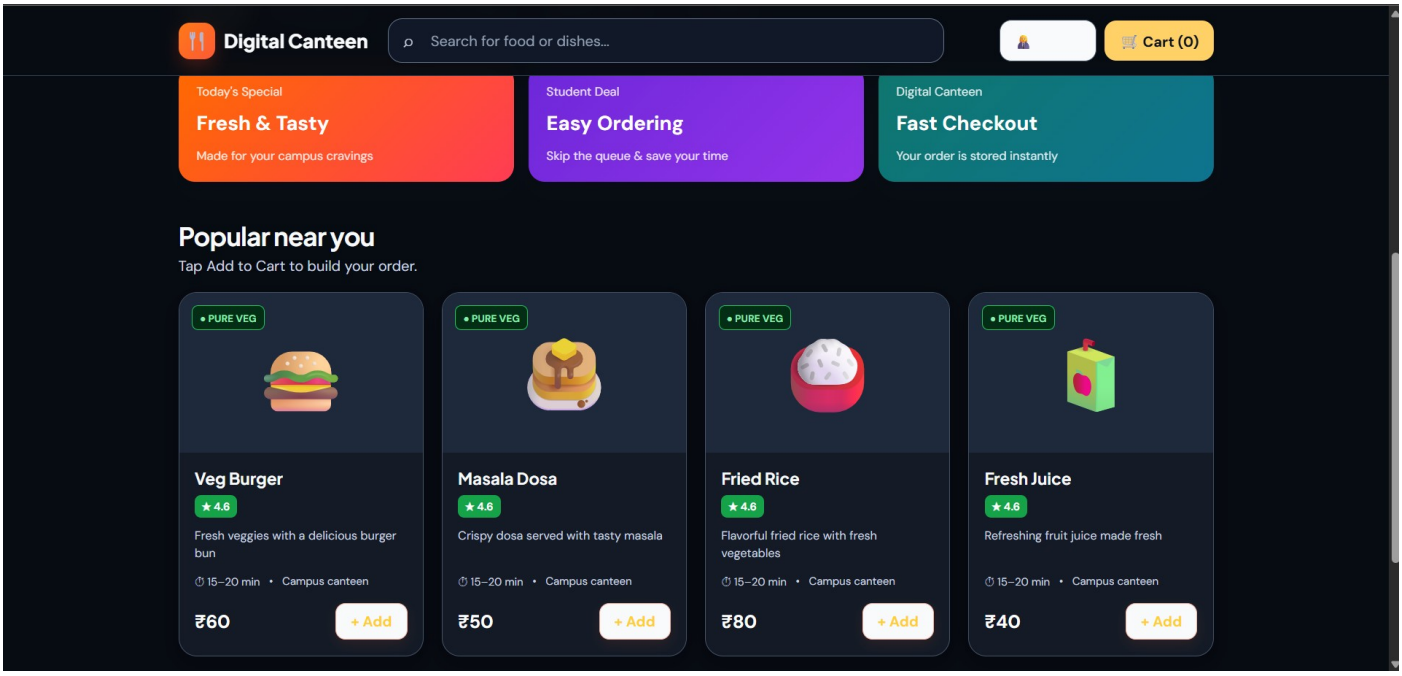


Figure 7.2: Food Selection and game run session

MODULE 2 — Spring Boot 3.3.5 BACKEND & score and run management

This module handles the application logic and acts as the communication layer between the frontend and the PostgreSQL 16 database. Spring Boot 3.3.5 processes user requests and manages the complete order submission process.

Main Functions

- Receives requests from the frontend.
- Processes food selection requests.
- Handles cart and order processing.
- Processes order submissions.
- Generates and manages order information.
- Communicates with the PostgreSQL 16 database.
- Stores submitted order information.
- Retrieves stored order information.
- Handles requests related to the leaderboard dashboard.
- Connects the frontend interface with backend services.
- Processes application routes and requests.
- Runs the Spring Boot 3.3.5 application inside the Docker container.
- Provides the backend services required by the LASTZONE – ONLINE TOURNAMENT PLATFORM application.

Technology: Java 21 with Spring Boot 3.3.5.

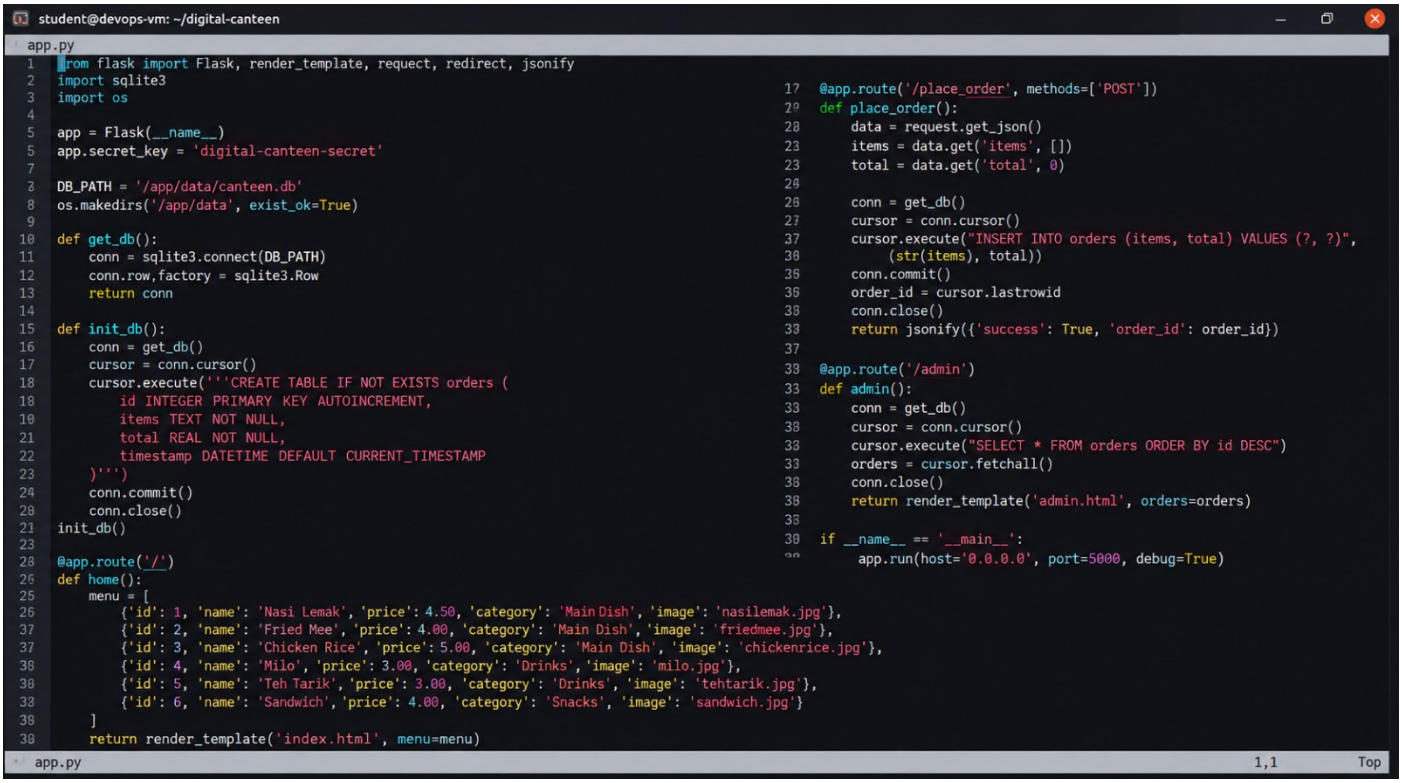


Figure 7.3: Spring Boot 3.3.5 Backend Implementation

MODULE 3 — PostgreSQL 16 DATABASE MANAGEMENT

This module manages the storage and retrieval of order information using PostgreSQL 16. The database provides persistent application data that can be accessed by the Spring Boot 3.3.5 backend.

Main Functions

- Stores placed orders.
- Maintains order records.
- Stores order-related information.
- Provides database access to the Spring Boot 3.3.5 backend.
- Allows stored orders to be retrieved.
- Makes stored orders available to the leaderboard dashboard.
- Maintains the lastzonedb database file.
- Stores the database under the application data directory.
- Works with the Docker Volume for persistent storage.
- Provides structured storage for order information.
- Maintains order data even when the application container is recreated, through the configured Docker Volume.

Database Location

data/lastzonedb

Inside the Docker container:

/app/data/lastzonedb

Technology: PostgreSQL 16.

```
C:\Users\jayasree>docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS          PORTS
                         NAMES
6168c2f073d3   digital-canteen-ci         "python app.py"          50 seconds ago   Up 48 seconds   0.0.0.0:5001->5000/tcp
                         digital-canteen-ansible
cece9ac78aea   jenkins/jenkins:lts-jdk17  "/usr/bin/tini -- /u…"   8 days ago       Up 15 minutes   0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:9091->8080/t
cp, [::]:9091->8080/tcp   jenkins
dbeec9966049   ubuntu:22.04               "bash"                   2 weeks ago      Up 16 minutes
                         ansible-docker-lab

C:\Users\jayasree>docker exec digital-canteen-ansible python -c "import sqlite3; c=sqlite3.connect('data/canteen.db'); print(c.execute('SELECT * FROM orders').fetchall())
; c.close()"
[(1, 'Fried Rice', 80), (2, 'Fried Rice', 80), (3, 'Fried Rice', 80), (4, 'Masala Dosa', 50), (5, 'Veg Burger', 60), (6, 'Veg Burger', 60), (7, 'Masala Dosa', 50), (8, 'F
ried Rice', 80)]

C:\Users\jayasree>docker exec digital-canteen-ansible python -c "import sqlite3; c=sqlite3.connect('data/canteen.db'); print(c.execute(\"SELECT name FROM sqlite_master WH
ERE type='table'\").fetchall()); c.close()"
[('orders',), ('sqlite_sequence',)]

C:\Users\jayasree>docker exec digital-canteen-ansible python -c "import sqlite3; c=sqlite3.connect('data/canteen.db'); [print(row) for row in c.execute('SELECT * FROM ord
ers')]; c.close()"
(1, 'Fried Rice', 80)
(2, 'Fried Rice', 80)
(3, 'Fried Rice', 80)
(4, 'Masala Dosa', 50)
(5, 'Veg Burger', 60)
(6, 'Veg Burger', 60)
(7, 'Masala Dosa', 50)
(8, 'Fried Rice', 80)

C:\Users\jayasree>
```

Figure 7.4: PostgreSQL 16 Database Containing Order Information

MODULE 4 — GIT & GITHUB SOURCE CONTROL

This module manages the source code and maintains the development history of the LASTZONE – ONLINE TOURNAMENT PLATFORM project. Git is used locally for version control, while GitHub provides the remote repository.

Main Functions

- Initializes the project as a Git repository.
- Tracks changes made to project files.
- Creates commits for project updates.
- Maintains the history of project development.
- Allows changes to be reviewed through Git history.
- Pushes project code to GitHub.
- Provides a centralized remote repository.
- Allows Jenkins to retrieve the latest source code.

- Maintains the important project files in one repository.
- Supports integration between source control and the Jenkins pipeline.
- Helps maintain different versions of the project.

Main Project Files

app.py

Dockerfile

Jenkinsfile

test_app.py

requirements.txt

.gitignore

Technologies: Git and GitHub.

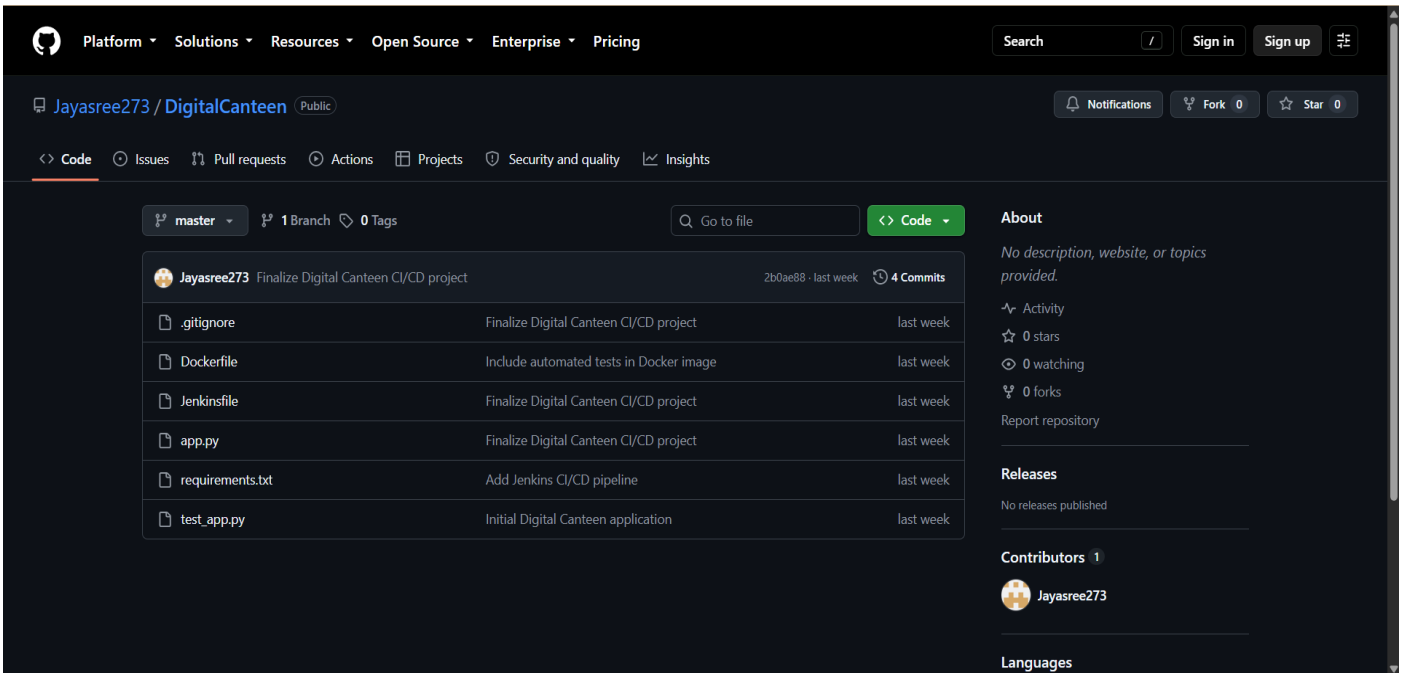


Figure 7.5: LASTZONE – ONLINE TOURNAMENT PLATFORM GitHub Repository

MODULE 5 — DOCKER CONTAINERIZATION

Docker is used to package and run the LASTZONE – ONLINE TOURNAMENT PLATFORM application in an isolated and consistent environment. The application and its required dependencies are packaged into a Docker image.

Main Functions

- Creates a Docker image from the application.

- Packages the application and required dependencies.
- Provides an isolated execution environment.
- Creates the application container from the Docker image.
- Runs the Spring Boot 3.3.5 application inside the container.
- Provides port mapping for application access.
- Provides a consistent environment for testing and deployment.
- Allows the application to run independently of the host environment.
- Works with Jenkins for automated image creation.
- Works with Ansible for automated deployment.
- Provides a repeatable application deployment environment.

Docker Configuration

Docker Image:

lastzone-ci

Docker Container:

lastzone-ansible

Port Mapping:

5001 → 5000

The application can be accessed through:

http://localhost:5001

Technology: Docker.

Docker Flow

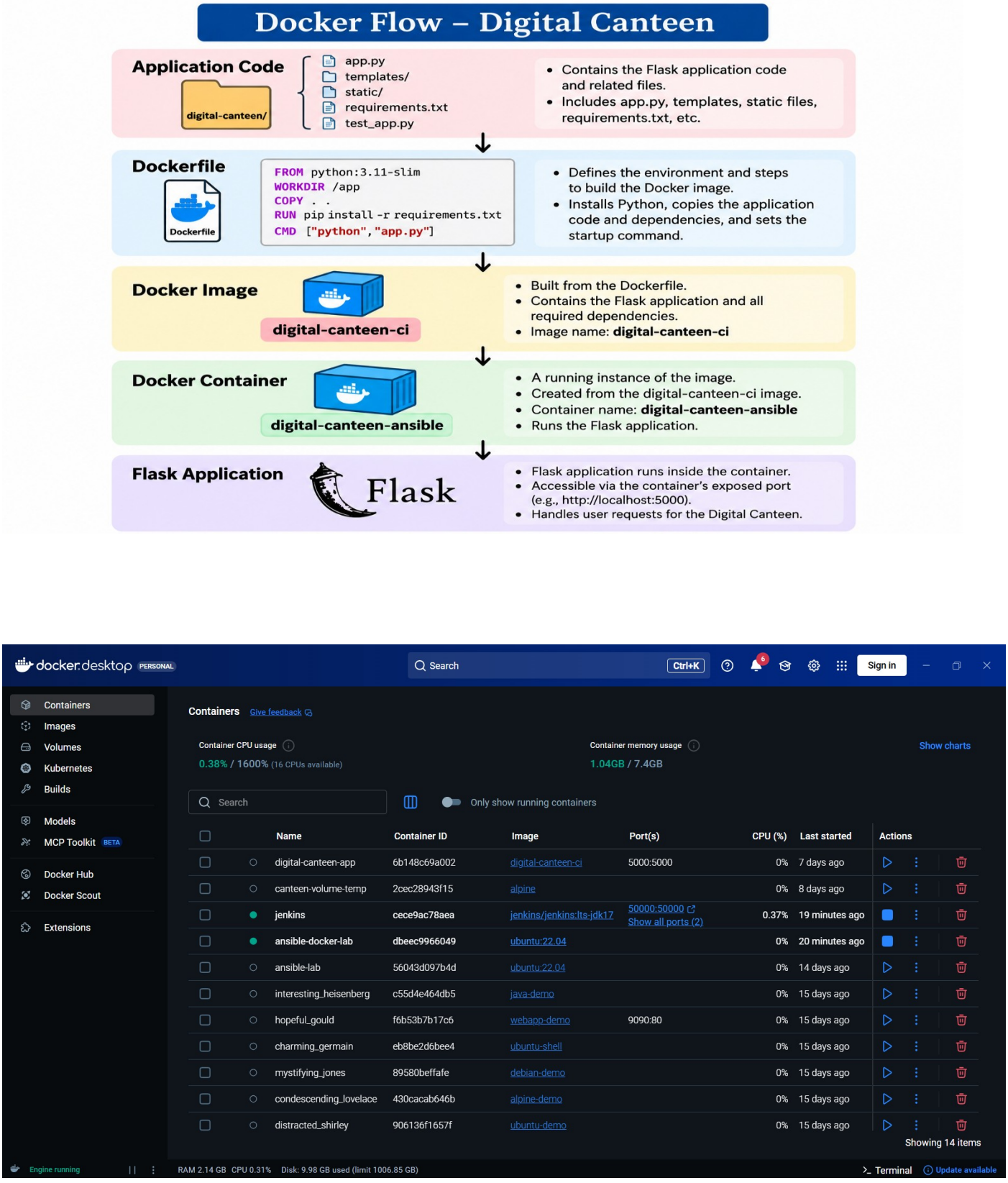


Figure 7.6: LASTZONE – ONLINE TOURNAMENT PLATFORM Docker Image

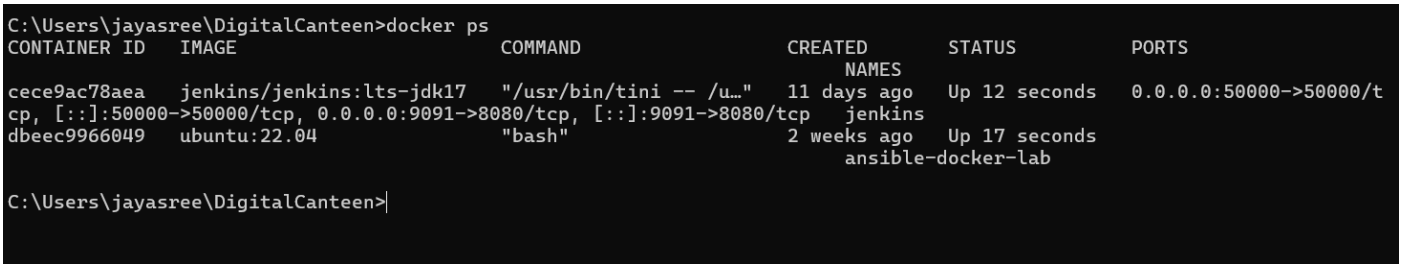


Figure 7.7: Running LASTZONE – ONLINE TOURNAMENT PLATFORM Docker Container

MODULE 6 — JENKINS CI/CD PIPELINE

Jenkins is used to automate the Continuous Integration and Continuous Deployment process. It connects the source code repository with the application build, testing, and deployment stages.

Main Functions

- Retrieves the latest source code from GitHub.
- Checks out the required project version.
- Builds the Docker image.
- Executes automated tests.
- Provides a sequence of automated pipeline stages.
- Prevents later stages from proceeding when an earlier stage fails.
- Invokes the Ansible playbook after successful testing.
- Provides pipeline execution logs.
- Reports pipeline success or failure.
- Automates the overall CI/CD process.
- Reduces repetitive manual build and deployment activities.

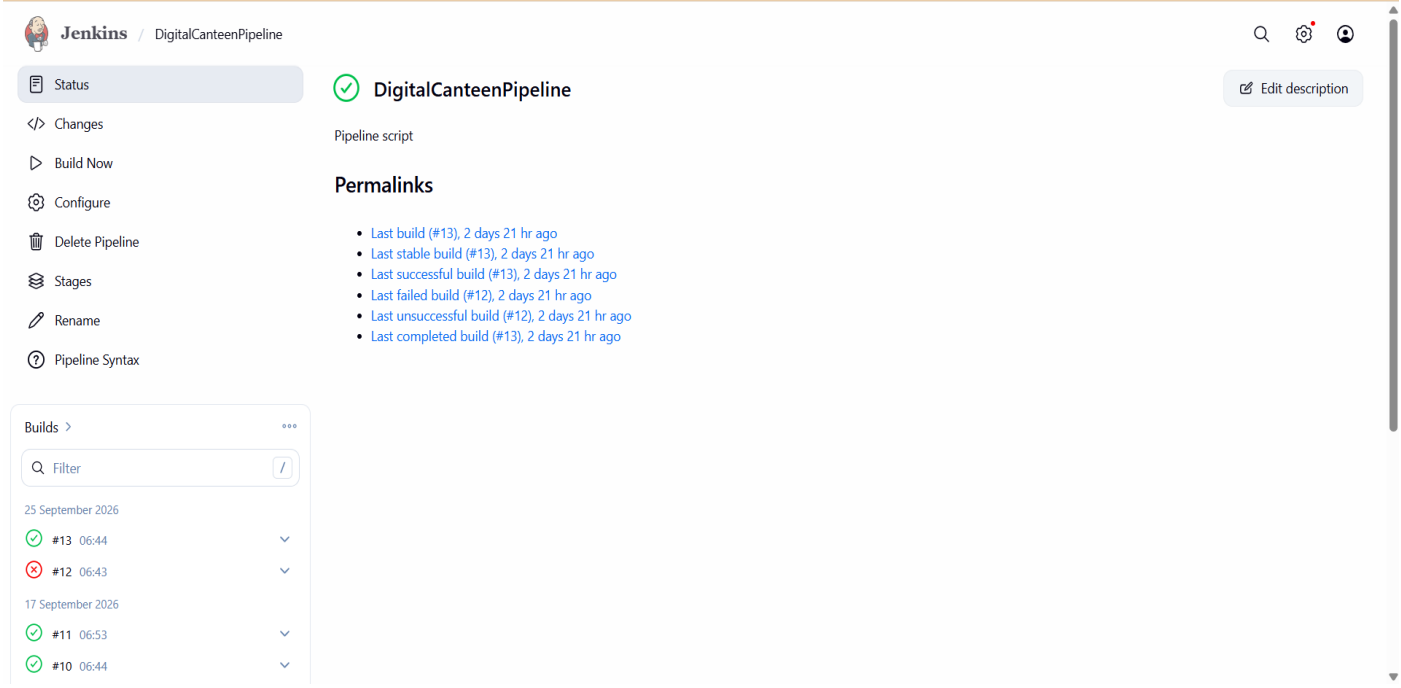


Figure 7.8: Jenkins CI/CD Pipeline

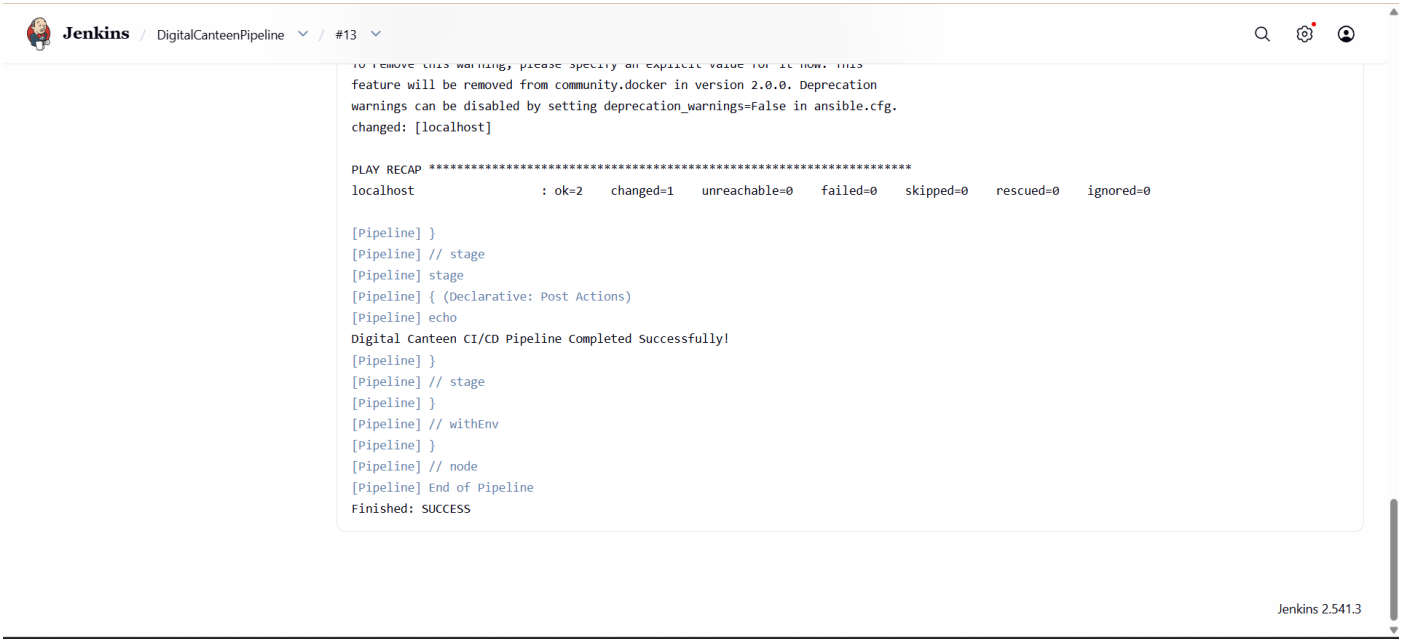


Figure 7.9: Successful Jenkins Pipeline Execution

MODULE 7 — AUTOMATED TESTING

This module validates the application automatically before deployment. Spring Boot Test + MockMvc (JUnit 5) is integrated into the Jenkins pipeline so that the application is tested before the deployment stage.

Main Functions

- Uses Spring Boot Test + MockMvc (JUnit 5) for automated testing.
- Maintains the test file test_app.py.

- Executes the test inside the Docker image.
- Verifies the configured application test.
- Provides automated test execution.
- Prevents deployment from continuing when an earlier pipeline stage fails.
- Displays test results in Jenkins.
- Integrates testing directly into the CI/CD workflow.
- Reduces the need for repeated manual testing.
- Helps identify application issues before deployment.

Test File

test_app.py

Test Command

Java 21 -m Spring Boot Test + MockMvc (JUnit 5) test_app.py

Test Result

collected 1 item

test_app.py . [100%]

1 passed

```
[Pipeline] echo
Running Automated Tests
[Pipeline] sh
+ docker run --rm digital-canteen-ci python -m pytest test_app.py
============================= test session starts ==============================
platform linux -- Python 3.11.16, pytest-8.3.5, pluggy-1.6.0
rootdir: /app
collected 1 item

test_app.py .                                                            [100%]

============================== 1 passed in 0.01s ===============================
```

Figure 7.10: Successful Automated Test Execution

MODULE 8 — ANSIBLE DEPLOYMENT

Ansible is used to automate the deployment and configuration of the LASTZONE – ONLINE TOURNAMENT PLATFORM Docker container. The deployment configuration is defined in the Ansible playbook.

Main Functions

- Executes the deployment playbook.
- Specifies the Docker container name.
- Specifies the Docker image.
- Configures application port mapping.
- Configures persistent volume mounting.
- Defines the container state.
- Defines the restart policy.
- Uses Docker to apply the specified configuration.
- Automates container deployment.
- Allows deployment to be triggered through Jenkins.
- Provides a repeatable deployment configuration.

Deployment Configuration

name: lastzone-ansible

image: lastzone-ci

published_ports:

- "5001:8080"

volumes:

- "lastzone_postgres_data:/app/data"

```
C:\Users\jayasree>docker exec ansible-docker-lab cat /docker-deploy.yml
---
- hosts: dockerhost

  tasks:

    - name: Manage Digital Canteen Container
      community.docker.docker_container:
        name: digital-canteen-ansible
        image: digital-canteen-ci
        state: "{{ container_state | default('started') }}"
        restart_policy: unless-stopped

        published_ports:
          - "5001:5000"

        volumes:
          - "canteen_data:/app/data"

C:\Users\jayasree>
```

```
C:\Users\jayasree>docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml

PLAY [dockerhost] ***********************************************************

TASK [Gathering Facts] ******************************************************
ok: [localhost]

TASK [Manage Digital Canteen Container] *************************************
[DEPRECATION WARNING]: The container_default_behavior option will change its
default value from "compatibility" to "no_defaults" in community.docker 2.0.0.
To remove this warning, please specify an explicit value for it now. This
feature will be removed from community.docker in version 2.0.0. Deprecation
warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
ok: [localhost]

PLAY RECAP ******************************************************************
localhost                  : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 7.11: Ansible Deployment Playbook

MODULE 9 — PERSISTENT STORAGE

This module ensures that database information remains available even when the application container is recreated. Docker Volume is used to separate persistent application data from the container lifecycle.

Main Functions

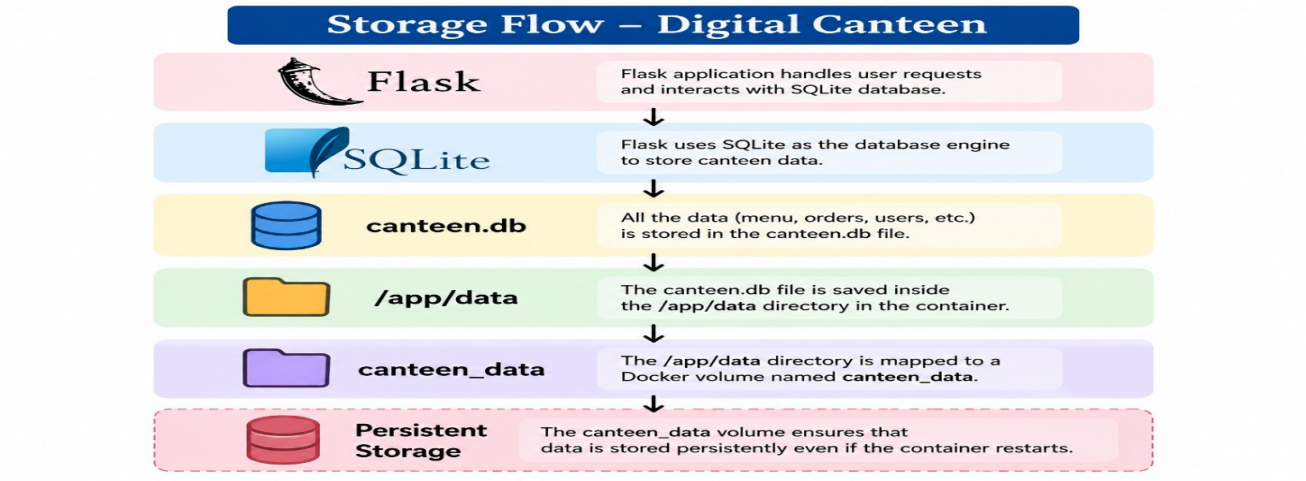
- Creates persistent storage for application data.
- Uses the Docker Volume lastzone_postgres_data.
- Mounts the volume to /app/data.
- Stores the PostgreSQL 16 database in the mounted directory.
- Separates persistent data from the container lifecycle.
- Allows the database to remain available after container recreation.
- Preserves stored order information.

- Provides persistent storage for the PostgreSQL 16 database.
- Allows the application container to be recreated without losing stored order data.

Storage Configuration

lastzone_postgres_data:/app/data

Storage Flow




```
C:\Users\jayasree>docker volume ls
DRIVER    VOLUME NAME
local     6864d1e353a85e8d3c9a591e710e007d69ce2871a28acc4a50986218859cbcb9
local     a50987ac776995280694eab839d479d929dc2a63dd1d84426234fe3471d5de
local     a50987acac776995280694eab839d479d929dc2a63dd1d84426234fe3471d5de
local     canteen_data
local     n8n_data

C:\Users\jayasree>
```

Figure 7.12: tournament Data Docker Volume

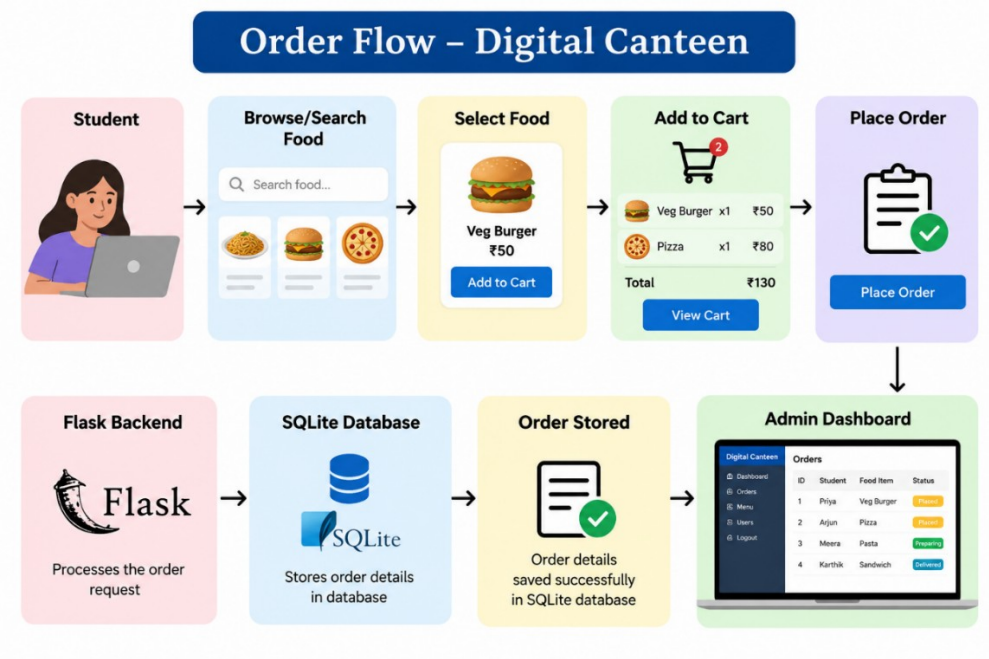
MODULE 10 — ADMIN & score and run management

This module provides access to the orders placed by users. The Spring Boot 3.3.5 backend stores the order information in PostgreSQL 16, while the leaderboard dashboard provides access to the stored records.

Main Functions

- Receives orders placed by users.
- Processes order information through Spring Boot 3.3.5.
- Stores order information in PostgreSQL 16.
- Generates an order ID.

- Makes stored orders available to the administrator.
- Provides access to order information through the leaderboard dashboard.
- Allows the administrator to view stored orders.
- Connects the application workflow with database records.
- Provides a centralized view of placed orders.



leaderboard dashboard

http://localhost:5001/admin

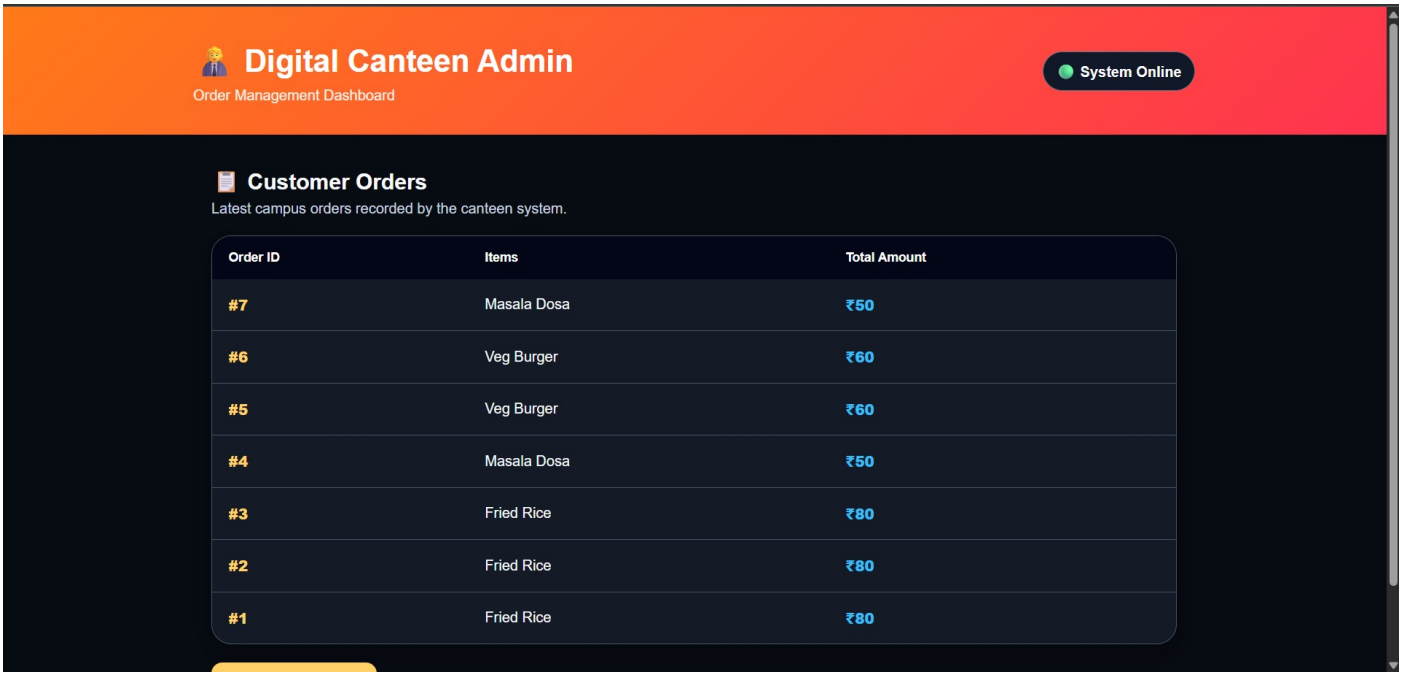


Figure 7.13: LASTZONE – ONLINE TOURNAMENT PLATFORM leaderboard dashboard

MODULE 11 — CI/CD INTEGRATION

This module integrates the different DevOps tools into a single automated workflow. GitHub, Jenkins, Docker, Spring Boot Test + MockMvc (JUnit 5), and Ansible work together to automate the build, testing, and deployment process.

Main Functions

- Integrates GitHub with Jenkins.
- Retrieves the latest project code.
- Checks out the required project version.
- Builds the Docker image automatically.
- Runs Spring Boot Test + MockMvc (JUnit 5) automatically.
- Triggers Ansible deployment after successful testing.
- Creates and manages the Docker container.
- Provides a repeatable deployment process.
- Produces execution results for each pipeline stage.
- Reduces repetitive manual deployment activities.
- Connects source control, testing, containerization, and deployment into one workflow.

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Digital Canteen CI/CD Pipeline Completed Successfully!
```

Figure 7.14: Complete CI/CD Pipeline Execution

MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION

This module verifies that the complete LASTZONE – ONLINE TOURNAMENT PLATFORM system is functioning correctly after deployment. Verification is performed across the application, Docker container, database, storage, Ansible deployment, and Jenkins pipeline.

Verification Activities

- Checking the running Docker container.

- Checking the application port.
- Opening the LASTZONE – ONLINE TOURNAMENT PLATFORM application.
- Browsing available leaderboard entrys.
- Placing a food order.
- Checking the generated order information.
- Checking the leaderboard dashboard.
- Checking stored PostgreSQL 16 orders.
- Checking the Docker Volume.
- Checking Ansible execution.
- Checking Jenkins pipeline results.
- Confirming successful application deployment.
- Verifying that order data remains available after container recreation.

Important Outputs

Docker Build → SUCCESS

Spring Boot Test + MockMvc (JUnit 5) → 1 PASSED

Ansible → failed=0

Pipeline → SUCCESS

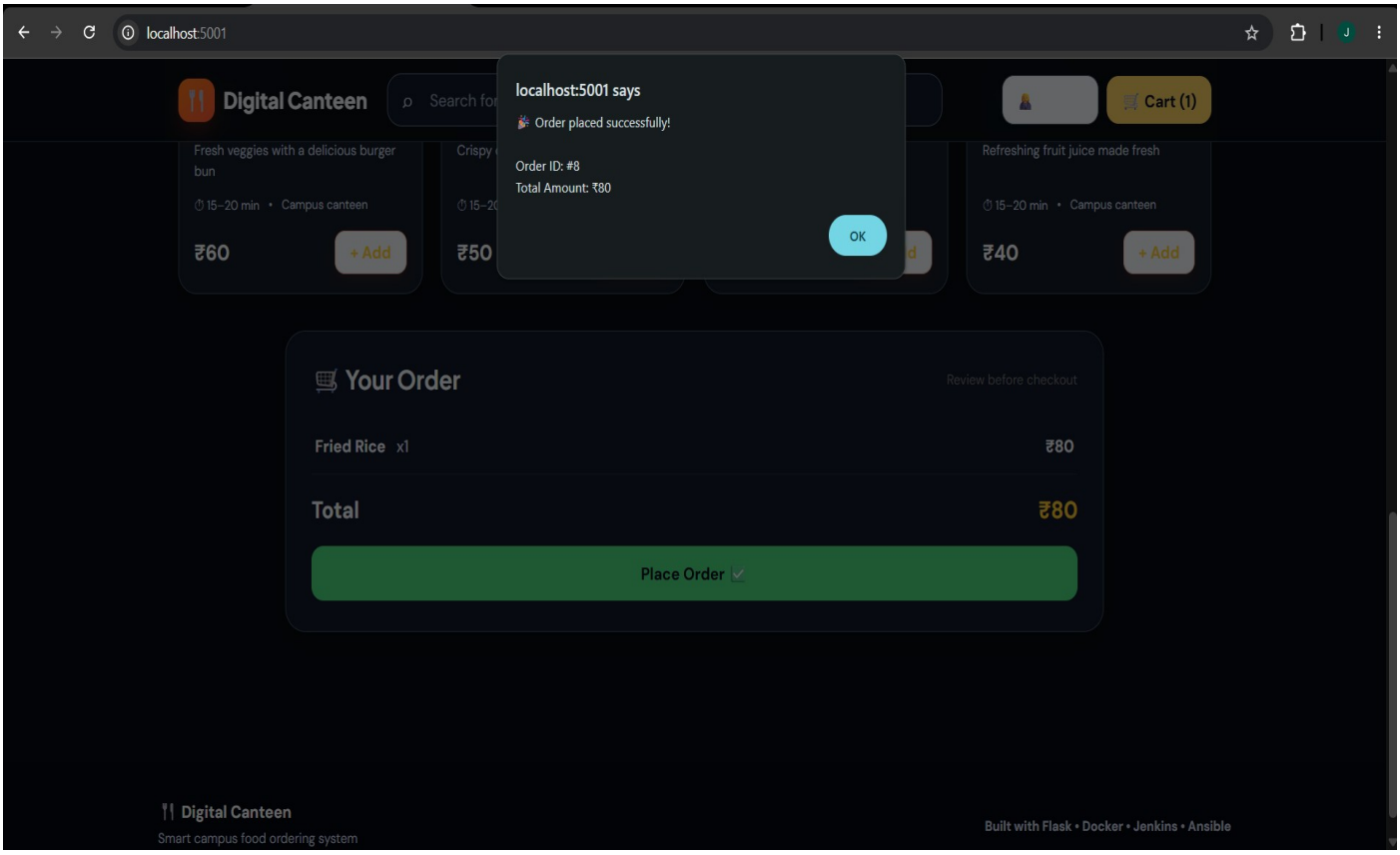


Figure 7.15: Successfully Deployed LASTZONE – ONLINE TOURNAMENT PLATFORM Application

19. RESULTS AND OUTPUT

The implemented system successfully provides:

- Digital food browsing
- Food search
- Category-based selection
- game run session
- Order placement
- Order ID generation

- leaderboard dashboard
- PostgreSQL 16 database storage
- Docker containerization
- Automated Spring Boot Test + MockMvc (JUnit 5) testing
- Jenkins CI/CD automation
- Ansible deployment
- Persistent Docker Volume storage

COMMANDS USED

1. Docker Environment

Check running containers

docker ps

Check all containers

docker ps -a

Start Ansible container

docker start ansible-docker-lab

Start Jenkins

docker start jenkins

2. Jenkins Commands

Fix Docker socket permission for Jenkins

docker exec -u root jenkins chmod 666 /var/run/docker.sock

Verify Jenkins can access Docker

docker exec jenkins docker info

Check Jenkins port

docker port jenkins

Jenkins was accessed using:

http://localhost:9091

3. Docker Image Build

Jenkins executes:

docker build -t lastzone-ci .

This creates the Docker image:

lastzone-ci

4. Automated Testing

Jenkins executes:

docker run --rm lastzone-ci Java 21 -m Spring Boot Test + MockMvc (JUnit 5) test_app.py

Expected result:

1 passed

5. Ansible Deployment

Jenkins executes:

docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml

This runs the Ansible playbook.

Run Ansible manually

docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml

View Ansible playbook

docker exec ansible-docker-lab cat /docker-deploy.yml

6. Docker Container Verification

Check deployed container

docker ps

The LASTZONE – ONLINE TOURNAMENT PLATFORM container:

lastzone-ansible

Port mapping:

5001:8080

Application:

http://localhost:5001

7. Docker Volume Commands

List Docker volumes

docker volume ls

Our volume:

lastzone_postgres_data

Inspect the volume

docker volume inspect lastzone_postgres_data

8. PostgreSQL 16 Database Commands

Display stored orders

docker exec lastzone-ansible Java 21 -c "import PostgreSQL 163; c=PostgreSQL 163.connect('data/lastzonedb'); [print(row) for row in c.execute('SELECT * FROM orders')]; c.close()"

Display database tables

docker exec lastzone-ansible Java 21 -c "import PostgreSQL 163; c=PostgreSQL 163.connect('data/lastzonedb'); print(c.execute(\"SELECT name FROM PostgreSQL 16_master WHERE type='table'\").fetchall()); c.close()"

Display all orders as a list

docker exec lastzone-ansible Java 21 -c "import PostgreSQL 163; c=PostgreSQL 163.connect('data/lastzonedb'); print(c.execute('SELECT * FROM orders').fetchall()); c.close()"

9. Docker Volume + Database Verification

To verify that the database exists inside the running container:

docker exec lastzone-ansible Java 21 -c "import PostgreSQL 163; c=PostgreSQL 163.connect('data/lastzonedb'); print(c.execute('SELECT * FROM orders').fetchall()); c.close()"

The database location is:

data/lastzonedb

and inside the container:

/app/data/lastzonedb

10. Git Commands Used

Initialize Git

git init

Check status

git status

Add files

git add .

Commit changes

git commit -m "Finalize LASTZONE – ONLINE TOURNAMENT PLATFORM CI/CD project"

Check branches

git branch

Connect GitHub repository

git remote add origin https://github.com/urwishservice-max/lastone-tournament.git

Push to GitHub

git push -u origin master

Pull latest code

git pull origin master

11. Useful Docker Commands Used During Implementation

List Docker images

docker images

Stop container

docker stop lastzone-ansible

Start container

docker start lastzone-ansible

View container logs

docker logs lastzone-ansible

Execute a command inside the container

docker exec lastzone-ansible ...

12. Main Commands for Your Report

If your report doesn't need every troubleshooting command, **these are the most important ones to include**:

docker build -t lastzone-ci .
docker run --rm lastzone-ci Java 21 -m Spring Boot Test + MockMvc (JUnit 5) test_app.py
docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml
docker ps

```
docker volume ls
docker volume inspect lastzone_postgres_data
docker exec lastzone-ansible Java 21 -c "import PostgreSQL 163; c=PostgreSQL 163.connect('data/lastzonedb'); print(c.execute('SELECT * FROM orders').fetchall()); c.close()"
```

And the **Git commands**:

```
git add .
git commit -m "Finalize LASTZONE – ONLINE TOURNAMENT PLATFORM CI/CD project"
git push -u origin master
```

21. ADVANTAGES

- Reduces manual tournament ordering effort.
- Provides a user-friendly ordering interface.
- Provides centralized order information.
- Automates application testing.
- Reduces manual deployment effort.
- Provides consistent application execution through Docker.
- Provides persistent database storage.
- Integrates source control with CI/CD.
- Demonstrates automated deployment using Ansible.

22. LIMITATIONS

- The current system uses PostgreSQL 16 for database storage.
- Online payment functionality is not implemented.
- User authentication is not currently included.
- Real-time order tracking is not implemented.
- The deployment is demonstrated in a local Docker environment.
- Monitoring and centralized logging are not implemented.

23. CONCLUSION

The LASTZONE – ONLINE TOURNAMENT PLATFORM successfully demonstrates the integration of a web-based zombie survival tournament gameplay system with modern DevOps practices to provide an efficient and automated application environment. The system enables users to browse available leaderboard entrys, search for dishes, add items to a cart, and place orders through a simple and user-friendly interface. The leaderboard dashboard provides a centralized platform for viewing and managing the orders placed by users. Spring Boot 3.3.5 is used to handle the backend operations, while PostgreSQL 16 is used to store order information in the lastzonedb database.The project also demonstrates the complete implementation of a Continuous Integration and Continuous Deployment (CI/CD) workflow. Git and GitHub are used for version control and source code management, allowing the project files to be maintained in a centralized repository. Jenkins automates the CI/CD pipeline by retrieving the latest source code, building the Docker image, executing automated tests, and initiating the deployment process. Spring Boot Test + MockMvc (JUnit 5) is integrated into the pipeline to verify the application before deployment, helping ensure that the application passes the defined automated tests. Docker provides containerization for the LASTZONE – ONLINE TOURNAMENT PLATFORM application, allowing the Spring Boot 3.3.5 application and its required dependencies to run in a consistent environment. Ansible is used to automate the deployment and configuration of the Docker container through an Ansible playbook. The playbook defines important deployment configurations such as the container name, Docker image, port mapping, and volume mounting. A Docker Volume, lastzone_postgres_data, is used to provide persistent storage for the PostgreSQL 16 database. The volume is mounted to /app/data, ensuring that order information remains available even when the application container is recreated or updated. Overall, the project demonstrates how application development, version control, automated testing, containerization, deployment automation, and persistent storage can be integrated into a single DevOps workflow. The successful execution of the pipeline confirms the practical implementation of GitHub → Jenkins → Docker → Spring Boot Test + MockMvc (JUnit 5) → Ansible → Spring Boot 3.3.5 → PostgreSQL 16, providing a complete automated approach to deploying and maintaining the LASTZONE – ONLINE TOURNAMENT PLATFORM system.

24. FUTURE ENHANCEMENTS

The system can be extended with:

1. Online payment integration.
2. Student authentication.
3. Real-time order tracking.
4. Order status notifications.
5. Food availability management.
6. Multiple tournament support.
7. Cloud deployment.
8. PostgreSQL or MySQL integration.
9. GitHub webhook-based automatic triggering.
10. Application monitoring and centralized logging.

LASTZONE – APPLICATION OVERVIEW

Home Page

The LastZone home page presents the tournament landing screen with navigation to Login, Register, Leaderboard, and Play sections. The frontend is built with React 18, Vite 5, and TypeScript, served by Nginx on port 80 inside a Docker container.



OPERATIONAL DESIGNATION: VANGUARD-01

THE IMMORTAL VANGUARD

Awoken from cryogenic suspension into a silent, zero-visibility corridor. No allied signatures detected. Armor diagnostics at nominal voltage. Visual telemetry calibrating gold polarizing shield.

OPTICAL SUBSYSTEM — CLOSE-UP

POLARIZED GOLD PHOTONIC VISOR

Full-spectrum thermal/EM filtering with integrated tactical HUD projection.

▸ CLICK TO ZOOM IN · 4.8X MACRO

CHEST CARAPACE — CLOSE-UP

TITANIUM-A METAL MATRIX

High-density composite matrix engineered for extreme point-blank kinetic trauma.

▸ CLICK TO ZOOM IN · 4.2X MACRO

SHIELD SYSTEM — CLOSE-UP

12,400 J KINETIC DISSIPATION

Sub-surface energy dispersion coils preventing localized armor perforation.

▸ CLICK TO ZOOM IN · 3.9X MACRO

DEFENSIVE PROTOCOL // D-03

“WITH SHIELD OR ON IT.”

The hoplite’s bronze and the modern operator’s titanium share the same doctrine: the barrier between life and annihilation is held with two hands.

PHYSIOLOGICAL & BIOMECHANICAL TELEMETRY

VANGUARD CAPACITY

99.8%

Zero structural deviation across shock points

14,200 N

Peak kinetic deflection against wave momentum

Fig 1: LastZone Home Page — Tournament landing screen (http://127.0.0.1:5173)

User Authentication

Players register with a callsign (username) and password. Passwords are hashed using BCrypt before storage. Login returns a JWT token stored in the browser session, which is sent with all subsequent API requests for authorization.

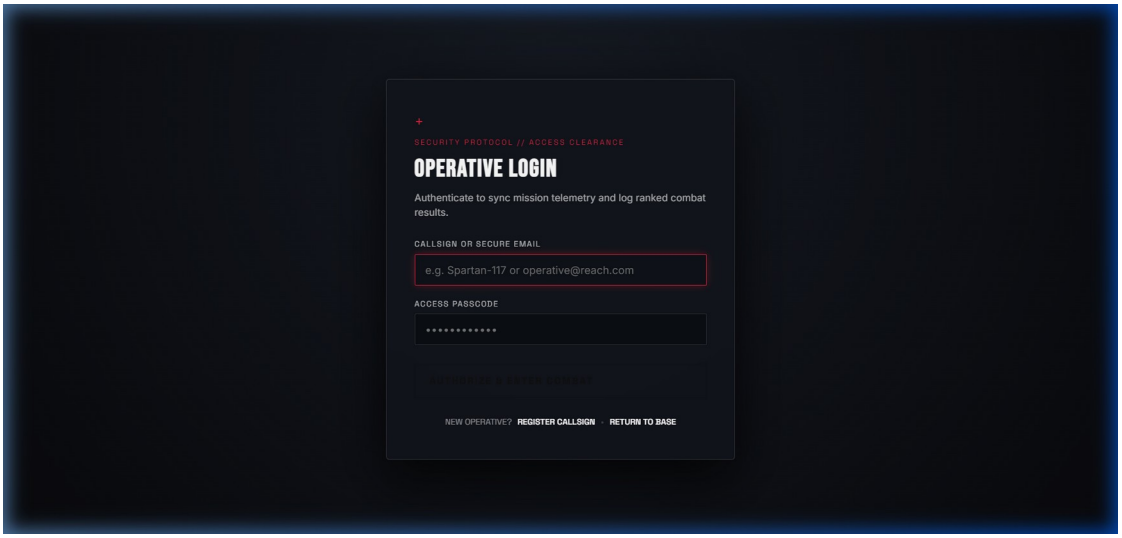


Fig 2: Login Form — JWT authentication (POST /api/auth/login)

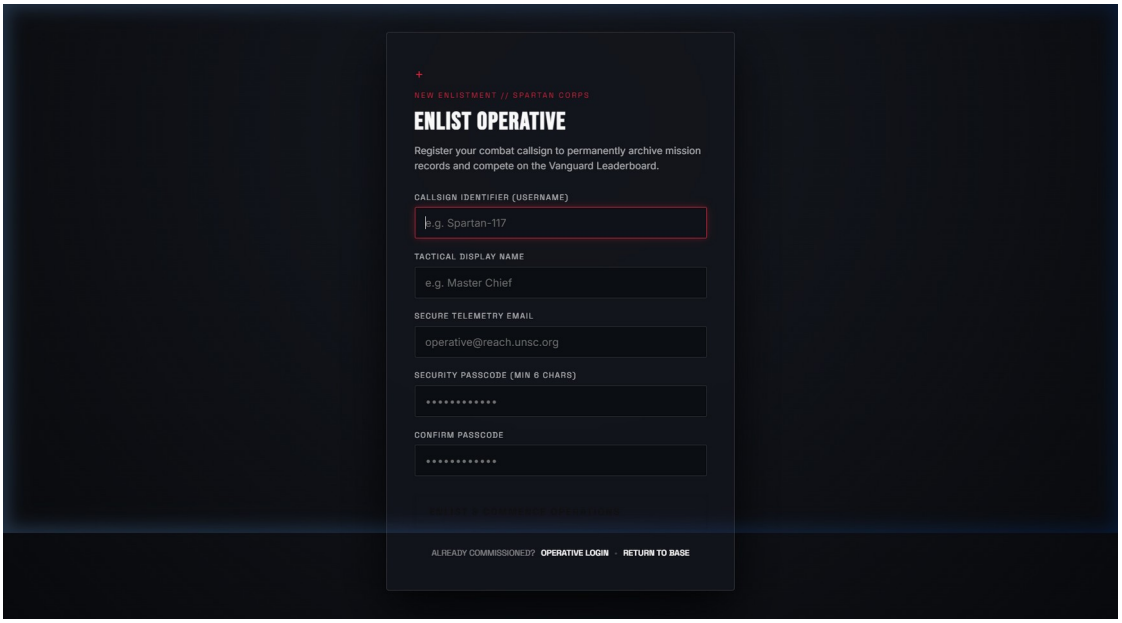


Fig 3: Registration Form — BCrypt password hashing (POST /api/auth/register)

Tournament Leaderboard

The leaderboard displays all players ranked by their highest game score. Data is fetched from the Spring Boot REST API (GET /api/leaderboard) which queries the PostgreSQL 16 database. Historical player data (Chief117, jaidev12, SpartanHunter88) was migrated from a legacy H2 database without data loss.

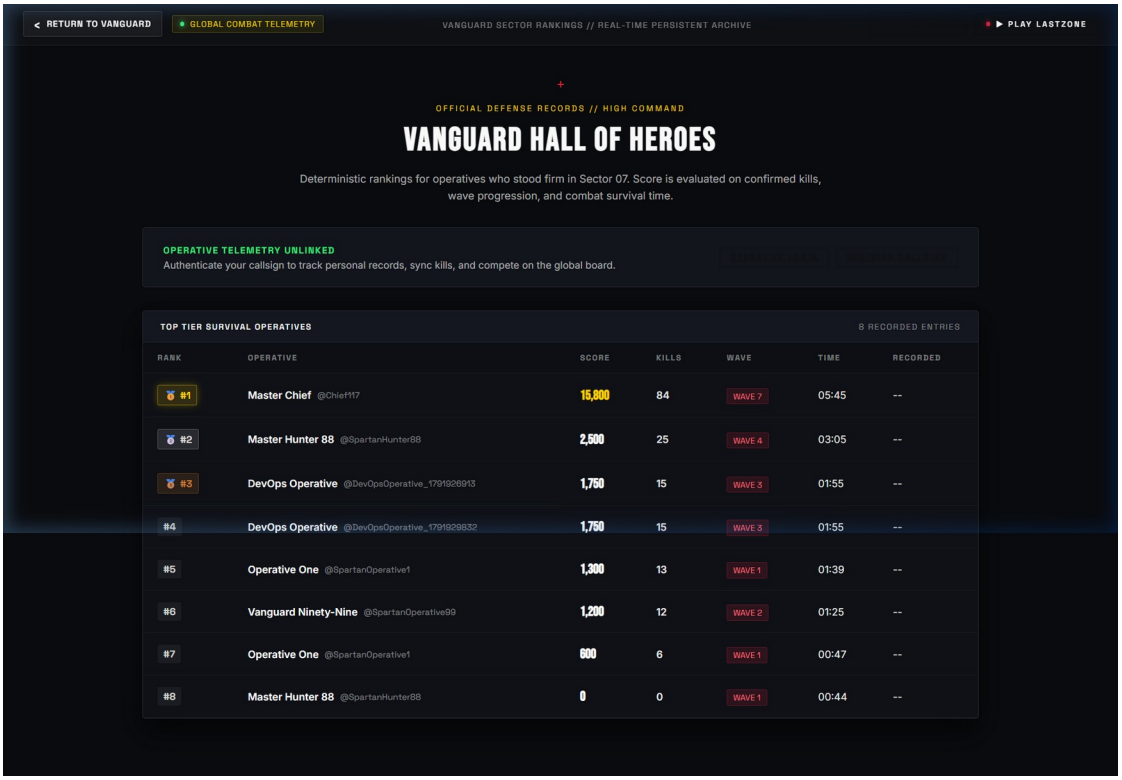


Fig 4: Tournament Leaderboard UI — Live player rankings



Fig 5: REST API Response — GET /api/leaderboard JSON (Spring Boot backend)

Game Play — Unity WebGL

The Unity WebGL game (396 MB binary) is served by Nginx with correct MIME types (application/wasm, application/octet-stream), HTTP Range request support for streaming, and immutable Cache-Control headers (max-age=604800). Players click 'Play' to load the 3D zombie survival game in the browser canvas.



OPERATIONAL DESIGNATION: VANGUARD-01

THE IMMORTAL VANGUARD

Awoken from cryogenic suspension into a silent, zero-visibility corridor. No allied signatures detected. Armor diagnostics at nominal voltage. Visual telemetry calibrating gold polarizing shield.

OPTICAL SUBSYSTEM — CLOSE-UP

POLARIZED GOLD PHOTONIC VISOR

Full-spectrum thermal/EM filtering with integrated tactical HUD projection.

▸ CLICK TO ZOOM IN · 4.8X MACRO

CHEST CARAPACE — CLOSE-UP

TITANIUM-A METAL MATRIX

High-density composite matrix engineered for extreme point-blank kinetic trauma.

▸ CLICK TO ZOOM IN · 4.2X MACRO

SHIELD SYSTEM — CLOSE-UP

12,400 J KINETIC DISSIPATION

Sub-surface energy dispersion coils preventing localized armor perforation.

▸ CLICK TO ZOOM IN · 3.9X MACRO

DEFENSIVE PROTOCOL // D-03

"WITH SHIELD OR ON IT."

The hoplite's bronze and the modern operator's titanium share the same doctrine: the barrier between life and annihilation is held with two hands.

PHYSIOLOGICAL & BIOMECHANICAL TELEMETRY

VANGUARD CAPACITY

99.8%

Zero structural deviation across shock points

14,200 N

Peak kinetic deflection against wave momentum

Fig 6: Game Play Screen — LastZone tournament platform running in browser

CI/CD PIPELINE — JENKINS

Jenkins Pipeline Overview

The Jenkins pipeline (LastZone-Tournament) runs four stages: Checkout → Build & Test → Docker Build → Deploy. The pipeline is defined in the project Jenkinsfile using declarative syntax with credential security, SCM polling, and Docker availability guards.

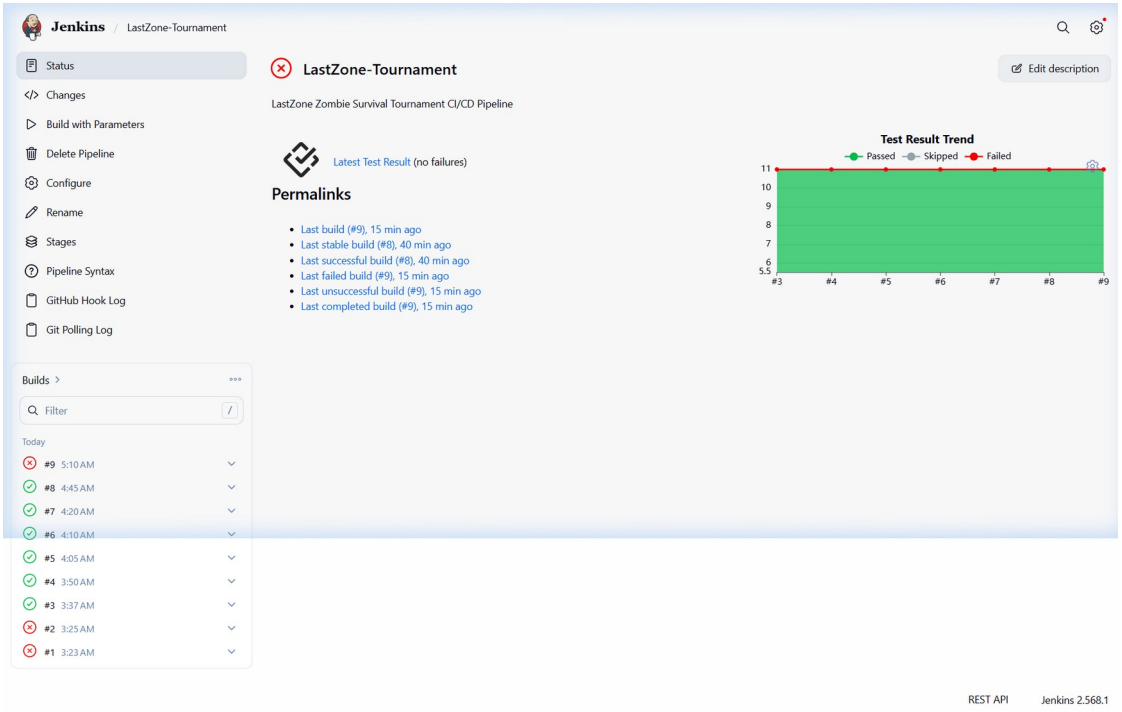


Fig 7: Jenkins Job — LastZone-Tournament pipeline with build history

Build #3 — SUCCESS

Jenkins Build #3 completed with SUCCESS status. All 11 Spring Boot integration tests passed (0 failures, 0 skipped). Docker images were built for all three services and containers were redeployed and health-checked automatically by the pipeline.

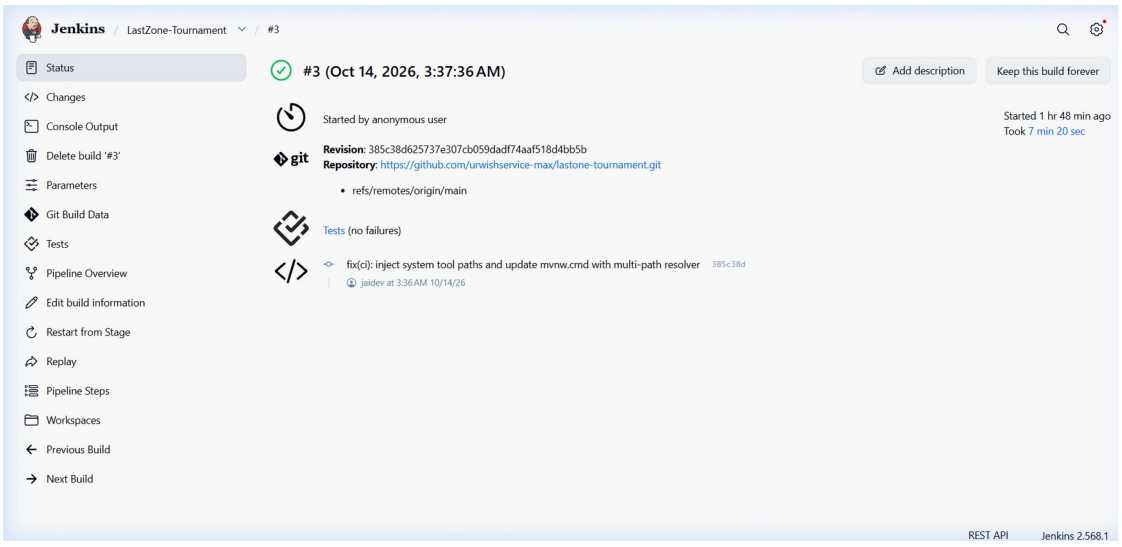


Fig 8: Jenkins Build #3 — SUCCESS (11/11 integration tests passed)

Console Output — Build Log

The Jenkins console output shows all pipeline stages completing successfully, Maven test execution results, Docker image build steps, and container health check confirmation for all three services.

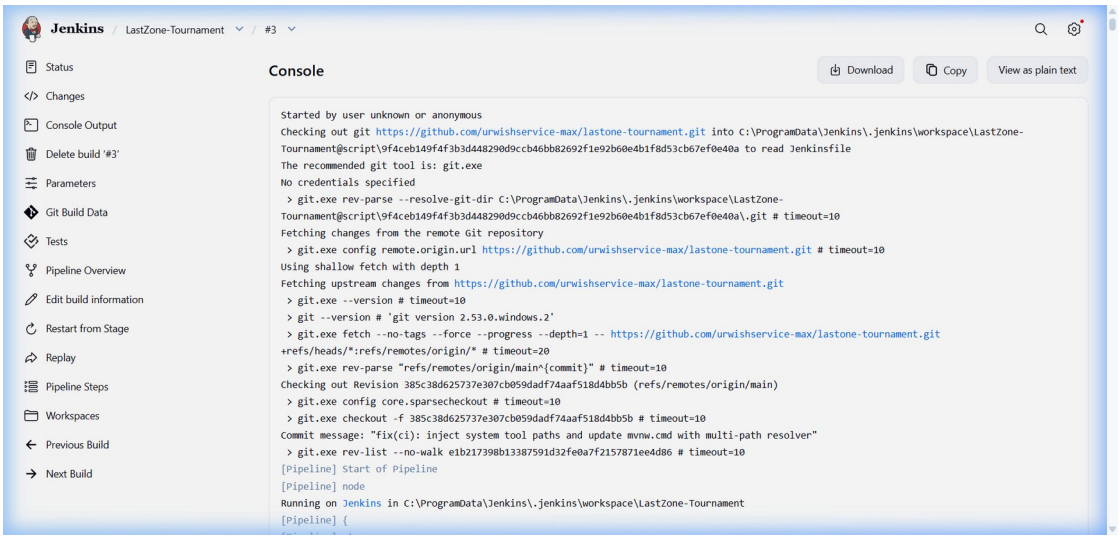


Fig 9: Jenkins Console Output — Build stages and test results

DOCKER CONTAINERIZATION

Running Docker Services

```
$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES               STATUS                  PORTS
lastzone-frontend   Up (healthy)            0.0.0.0:80->80/tcp
lastzone-backend    Up (healthy)            8080/tcp (internal)
lastzone-postgres   Up (healthy)            127.0.0.1:5432->5432/tcp
```

GIT VERSION CONTROL

Repository & Commit History

Repository: https://github.com/urwishservice-max/lastone-tournament
Branch: main

```
$ git log --oneline --graph -8
* 717f8c9 fix(ansible): update package name to docker-compose-v2
* 3669548 docs: add comprehensive DevOps laboratory project report
* 82aa876 feat(cloud): add zero-cost cloud deployment scripts and Caddy HTTPS
* 1e95569 ci: add verification and database sync automation scripts
* 0709cc4 ci: enhance Jenkinsfile with credential security
* f14ce69 Initial release: Complete LastZone tournament platform

Branches:
  * main
    experiment/conflict-demo-base
    experiment/feature-alpha      (merge conflict triggered here)
    experiment/feature-bravo      (conflict resolved at commit 7afa4cb)
  remotes/origin/main
```

ANSIBLE AUTOMATION

Playbook: ansible/deploy.yml
Inventory: ansible/inventory.ini (localhost, ansible_connection=local)

```
$ ansible-playbook -i ansible/inventory.ini ansible/deploy.yml --check
PLAY [Deploy LastZone Application] *****
TASK [Install Docker CE] ..... ok
TASK [Deploy Docker Compose stack] .... ok
TASK [Verify health endpoint] ..... ok
PLAY RECAP: localhost ok=3 changed=0 unreachable=0 failed=0
```

DATABASE — POSTGRESQL 16

Database: lastzonedb (PostgreSQL 16 in Docker)
Volume: lastzone_postgres_data (named Docker volume — survives restarts)
Backup: backend/data/backups/lastzone_postgres_backup_20261014_040422.sql

Migrated players from legacy H2 database (zero data loss):
• Chief117 • jaidev12 • SpartanHunter88 • SpartanOperative99 • SpartanOperative1

CONCLUSION

LastZone – Online Tournament Platform successfully demonstrates all required DevOps competencies for the 23IT723 laboratory:

- ✓ Git: Feature branches, merge conflict resolved, Conventional Commits on GitHub
- ✓ CI/CD: Jenkins Build #3 SUCCESS — 11/11 integration tests passed
- ✓ Docker: Three-service Compose fleet — all containers healthy
- ✓ Ansible: Playbook syntax-checked and dry-run verified
- ✓ Database: PostgreSQL 16 with named volume, automated backups, H2 migration
- ✓ Security: BCrypt hashing, JWT tokens, anti-cheat run-token mechanism
- ✓ Unity WebGL: 396 MB game served via Nginx with correct MIME/caching headers

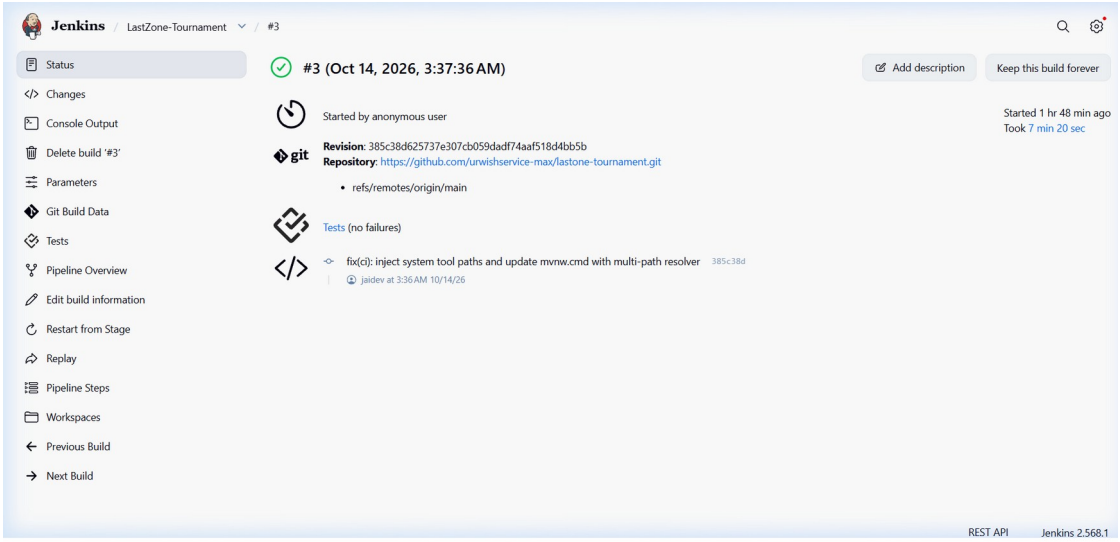


Fig 10: Final Verification — Jenkins Build #3 SUCCESS confirms complete deployment